

ECOM EXPERTS

# Ecom passkeytotp

Manual de usuario, tecnico y de desarrollo

---

ecom\_passkeytotp 1.0.7 · 07/09/2026

## Contenido

### 1. Qué hace el módulo

### 2. Instalación

Si has cambiado de un módulo de 2FA anterior

### 3. Configuración global (pestaña 2FA Config)

### 4. Acceso ADMIN — paso a paso

#### 4.1 Configurar TOTP de un empleado

#### 4.2 Configurar Passkey de un empleado

#### 4.3 Login en admin con Passkey

#### 4.4 Borrar passkeys de admin

### 5. Acceso CLIENTE — paso a paso

#### 5.1 Habilitar TOTP en cuenta de cliente

#### 5.2 Registrar Passkey de cliente

#### 5.3 Login cliente con Passkey

### 6. Recuperación

Empleado admin que perdió el dispositivo TOTP

Cliente que perdió TOTP

### 7. Logs

### 8. Desinstalación

#### 1. Resumen

#### 2. Estructura de archivos

#### 3. Esquema de base de datos

#### 4. Hooks

#### 5. Endpoint AJAX único

Acciones soportadas

### 6. Convenciones críticas (NO romper)

#### 6.1 Nombre de la clase principal con \_

#### 6.2 \_PS\_ADMIN\_DIR\_ antes de config.inc.php

#### 6.3 Login admin con passkey — campos cookie

#### 6.4 Login Admin en PrestaShop 9 (AJAX)

## 6.5 No usar @ para silenciar excepciones de Db

## 6.6 No override/, no controllers front nuevos

## 6.7 Smarty Syntax (Curly Braces)

### 7. Seguridad WebAuthn

### 8. Cifrado de secretos TOTP

### 9. Auto-reparación

### 10. Logs

### Extensibilidad

### Debugging

### Lógica de Login (Admin)

### Lógica de Login (Cliente)

### Criptografía

### Advertencia de Mantenimiento

### Reglas globales

### Requisitos PHP nativos (verificar en addnewfeatures() y mostrar warning si falta alguno)

### Librerías embebidas (a copiar dentro de vendor/internal/)

## 1. JWT/JWS/JWK toolkit — implementación propia

## 2. QR generator

## 3. ASN.1 / DER parser

### Estructura de carpetas

### Constantes de configuración nuevas (Configuration::set)

### Decisiones técnicas críticas

### Estado de Fase 2

### Decisiones congeladas (versión v2.0 del módulo)

## Rol del módulo

## Estándares y versiones

## Modos de uso

## Casos de uso v2.0

## Casos de uso v2.1+ (no se implementa ahora, pero la arquitectura los soporta)

## Selective disclosure

## Almacenamiento

Trust / confianza en issuers

Criptografía mínima

Compatibilidad

Reglas del proyecto que aplican

Riesgos conocidos y mitigaciones

Dependencias entre fases

Nombre comercial

Endpoints nuevos (preview de Fase 5/6)

Contrato JSON eudi/start (preview)

Quién hace qué

Glosario rápido

1. Lo que cambió entre PS8 y PS9 (fuente: devdocs oficial)

1.1 Hooks de login admin ELIMINADOS en PS9

1.2 Hooks que se mantienen

1.3 Hooks NUEVOS en PS9

1.4 Sesiones y cookies — cambio crítico

1.5 Clase Employee para Symfony Security

1.6 EmployeeProvider::refreshUser() — origen del "Tu sesión se ha cerrado por motivos de seguridad"

1.7 Para reaccionar al login en PS9 — patrón oficial

2. Por qué cada uno de mis intentos falló

3. Solución correcta arquitectónica para PS9

3.1 Estructura

3.2 Flujo de login admin TOTP en PS9

3.3 Flujo passkey admin en PS9

4. Pendiente — pasos concretos

5. Fuentes consultadas

Módulo de **doble factor (TOTP)** y **Passkeys (WebAuthn)** para PrestaShop 8.x. Cubre tanto el back office (empleados) como el front (clientes).

## 1. Qué hace el módulo

| Para quién      | Qué ofrece                                                                          |
|-----------------|-------------------------------------------------------------------------------------|
| Empleados admin | TOTP obligatorio en login + Passkey opcional (login sin contraseña, biometría/PIN). |
| Clientes front  | TOTP opcional + Passkey opcional + códigos de respaldo + recuperación por email.    |

Todos los secretos TOTP se guardan **cifrados (AES-256-GCM)** con clave derivada de `_NEW_COOKIE_KEY_` de PrestaShop.

## 2. Instalación

1. Sube el zip por *Módulos > Module Manager > Subir un módulo*.
2. Pulsa **Instalar**.
3. La pestaña aparece en *Parámetros avanzados > 2FA Config*.

Si la primera instalación queda a medias por algún motivo (timeout, caché), navega a cualquier página del back office: el módulo se **auto-repara** (recrea tablas faltantes y la pestaña) en el primer dispatcher.

### Si has cambiado de un módulo de 2FA anterior

Las filas viejas con secretos cifrados con otra `_COOKIE_KEY_` no se pueden descifrar. El módulo las detecta y las **borra automáticamente** la primera vez que las lee. El usuario afectado verá el botón "*Generate 2FA Key*" y se reinicia el setup.

### 3. Configuración global (pestaña 2FA Config)

| Opción                               | Qué hace                                                                                                             |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <b>Use generic Auth2</b>             | Si está ON, además del TOTP del empleado se acepta un código maestro genérico.<br>Default: <b>OFF</b> (recomendado). |
| <b>Generic Auth2</b>                 | El código genérico (cuando lo anterior está ON). Cambia el default por uno único antes de activarlo.                 |
| <b>Email alert</b>                   | Email donde llega el aviso cuando un empleado falla TOTP más de 3 veces.                                             |
| <b>Enable Passkeys for Admin</b>     | Habilita el botón <i>Login with Passkey</i> en la pantalla de admin.                                                 |
| <b>Enable Passkeys for Customers</b> | Habilita Passkey en login y área de cuenta del cliente.                                                              |
| <b>Exigir UV en admin</b>            | Si está ON, al loguear con passkey el navegador siempre pide PIN/biometría (estilo PayPal).                          |
| <b>Enable Debug Logs</b>             | Escribe en <code>modules/ecom_passkeytotp/logs/&lt;fecha&gt;_debug.log</code> .                                      |

## 4. Acceso ADMIN — paso a paso

### 4.1 Configurar TOTP de un empleado

1. Inicia sesión como ese empleado (email + contraseña).
2. Ve a *Parámetros avanzados > 2FA Config*.
3. Pulsa **Generate 2FA Key**.
4. Escanea el QR con Google Authenticator / Authy / Microsoft Authenticator.
5. La próxima vez que entres al admin, además de email + contraseña, te pedirá el código de 6 dígitos.

Cada empleado tiene **su propio** secret. Si lo regeneras, invalidas el anterior.

### 4.2 Configurar Passkey de un empleado

Pre-requisito: ya estás logueado en admin.

1. Al entrar al admin tras login, si no tienes passkey registrada, sale un prompt: "*¿Deseas registrar una llave de acceso?*". Acepta.
2. El navegador pide tu PIN / huella / cara / Windows Hello / Touch ID. Confirma.
3. Listo. La passkey queda asociada a este empleado y a este dispositivo.

Puedes registrar **hasta 10 passkeys** por empleado (varios dispositivos).

### 4.3 Login en admin con Passkey

1. En la pantalla de login, pulsa **Login with Passkey**.
2. Confirma con PIN/biometría.
3. Entrar directo, sin email ni contraseña.

Si pierdes el dispositivo: vuelve a entrar con email + contraseña + TOTP, y borra la passkey desde la pestaña del módulo.

### 4.4 Borrar passkeys de admin

Desde *Parámetros avanzados > 2FA Config*: - **Borrar mis passkeys** — borra solo las del empleado logueado.  
- **Borrar TODAS las passkeys de admin** — solo si eres super-admin. Acción irreversible.

## 5. Acceso CLIENTE — paso a paso

### 5.1 Habilitar TOTP en cuenta de cliente

1. Cliente entra a *Mi Cuenta*.
2. Sección "Doble factor / Passkeys" (la añade el módulo).
3. Pulsa *Activar TOTP*. Aparece QR + secret.
4. Escanea con la app, introduce el código actual para confirmar.
5. Se generan **10 códigos de respaldo** (descárgalos: cada uno funciona una sola vez).

A partir de ese momento, al loguear el cliente: 1. Introduce email + contraseña. 2. PrestaShop pide el código TOTP de 6 dígitos. 3. Si lo introduce bien, entra. Si no: - Puede usar un **código de respaldo**. - Puede pedir **recuperación por email** (link al correo, válido 10 minutos, máx. 3 por 24h).

### 5.2 Registrar Passkey de cliente

Pre-requisito: cliente ya logueado.

1. En *Mi Cuenta*, pulsa *Registrar Passkey*.
2. Confirma con PIN/biometría.
3. Próxima vez que vuelva a la tienda, podrá pulsar *Login con Passkey* en lugar de email+password.

### 5.3 Login cliente con Passkey

1. En la pantalla de login del front, pulsa **Login con Passkey**.
2. Confirma con biometría/PIN.
3. Sesión iniciada.

---

## 6. Recuperación

---

### Empleado admin que perdió el dispositivo TOTP

Sin acceso a la base de datos: **no se puede recuperar desde el front**. Otro super-admin debe entrar al servidor y, en la tabla `ps_employee_totp`, borrar la fila del empleado afectado. La próxima vez que entre, generará uno nuevo.

### Cliente que perdió TOTP

Tres caminos, en orden: 1. **Código de respaldo** — escribe uno de los 10 códigos generados. 2.

**Recuperación por email** — link válido 10 min, máximo 3 por 24h, máximo 5 intentos por link. 3. Contactar con la tienda para reseteo manual.

---

## 7. Logs

---

Si activas *Enable Debug Logs*: - Ruta: `modules/ecom_passkeytotp/logs/<YYYY-MM-DD>_debug.log` - Acceso bloqueado por `.htaccess` (no público). - **Nunca** escribe contraseñas, secrets, tokens, signatures ni `$_POST` / `$_COOKIE` — solo metadatos saneados.

---

## 8. Desinstalación

---

Desinstalar desde *Module Manager* elimina: - Todas las tablas del módulo. - La pestaña *2FA Config*. - Todas las claves de configuración del módulo.

**No quedan datos residuales.**

# Guía Técnica — ecom\_passkeytotp

---

Documentación de arquitectura, esquema de DB, hooks, clases y reglas críticas. Pensada para alguien que tenga que mantener o auditar el módulo.

---

## 1. Resumen

---

PrestaShop 8.x. Implementa:

- **TOTP (RFC 6238)** propio para empleados admin y clientes front.
- **Passkeys (WebAuthn)** propio (sin librerías de terceros), con verificación estricta de challenge / origin / rpIdHash / signCount / UP / UV.
- **Backup codes** (10 por cliente, sha256, single-use).
- **Recuperación por email** con rate-limit (3/24h) y límite de intentos (5/token).
- **Conectividad Gmartos**: Registro automático y verificación semanal de versiones en [modules.gmartos.es](https://modules.gmartos.es).

No usa `override/`. Todo va por hooks o controladores propios del módulo.

---

## 2. Estructura de archivos

```
ecom_passkeytotp/
├── ecom_passkeytotp.php      # Clase principal del módulo (Ecom_Passkeytotp)
├── ajax.php                 # Endpoint AJAX único (front + admin)
├── classes/
│   ├── EcomPasskeytotpCrypto.php  # AES-256-GCM con HKDF de _NEW_COOKIE_KEY_
│   ├── EcomPasskeytotpCsrf.php    # Token CSRF por cookie
│   ├── EcomPasskeytotpLog.php     # Logger saneado (no escribe secrets)
│   ├── EcomPasskeytotpWebAuthnHandler.php  # Verificación WebAuthn estricta
│   └── TwoFactorKey.php          # TOTP, backup codes, recovery
├── controllers/
│   ├── admin/
│   │   └── AdminEcomPasskeytotpController.php  # Pestaña "2FA Config"
│   └── front/
│       ├── info.php              # Página informativa CMS
│       └── webauthn.php          # Endpoint deprecado (410 Gone)
├── views/
│   ├── js/
│   │   ├── passkeys.js          # JS admin (login + register)
│   │   ├── passkeyfront.js      # JS cliente
│   │   └── login-totp.js        # Inyección TOTP en form admin
│   ├── css/                     # Estilos popup + UI
│   └── templates/
│       ├── admin/               # configure.tpl, employee2fa.tpl
│       ├── front/               # account_passkeys.tpl, info.tpl
│       └── hook/                # customer_login, totp_challenge, etc.
├── sql/install.php            # Reservado (usamos installDB() en clase)
├── upgrade/                  # Migraciones por versión
└── logs/                     # Debug logs (.htaccess Deny)
```

## 3. Esquema de base de datos

| Tabla (sin prefijo)       | Para qué                                                                                                                                                                  |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| employee_totp             | Secret TOTP del empleado admin (cifrado).                                                                                                                                 |
| employee_try              | Contador de intentos fallidos por empleado (alerta email a los 3+).                                                                                                       |
| customer_totp             | Secret TOTP del cliente (cifrado) + flag <code>active</code> .                                                                                                            |
| customer_totp_backup      | Códigos de respaldo (sha256, <code>used</code> , <code>used_at</code> ).                                                                                                  |
| customer_totp_recovery    | Tokens de recuperación por email con <code>attempts</code> , <code>ip_hash</code> , <code>ua_hash</code> .                                                                |
| ecom_passkeytotp_webauthn | Credenciales WebAuthn (admins y customers, <code>id_user</code> + <code>user_type</code> + <code>credential_id</code> + <code>public_key</code> + <code>counter</code> ). |
| ecom_passkeytotp_tokens   | Reservada (no usada actualmente).                                                                                                                                         |

`installDB()` usa `CREATE TABLE IF NOT EXISTS` con cada query envuelta en try/catch para que un fallo individual no aborte el resto. Los `ALTER TABLE` (migraciones) están protegidos por chequeo previo en `INFORMATION_SCHEMA` — son idempotentes.

## 4. Hooks

| Hook                                                                  | Para qué                                                                                                                                                |
|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>actionDispatcher</code>                                         | Auto-registro de hooks nuevos ( <code>ensureHooks()</code> ), auto-reparación de tablas y de Tab admin, re-hidratación del employee tras login passkey. |
| <code>actionAdminLoginControllerLoginBefore</code>                    | Procesa el login admin con passkey (escribe cookie admin, registra <code>EmployeeSession</code> , redirige).                                            |
| <code>actionAdminControllerSetMedia</code>                            | Inyecta <code>passkeys.js</code> y JS defs en TODAS las páginas admin.                                                                                  |
| <code>actionAdminLoginControllerSetMedia</code>                       | Inyecta también <code>login-totp.js</code> en la pantalla de login admin.                                                                               |
| <code>displayAdminAfterHeader</code> / <code>displayAdminLogin</code> | Inyectan defs adicionales según contexto (login vs logueado).                                                                                           |
| <code>actionFrontControllerSetMedia</code>                            | Inyecta <code>passkeyfront.js</code> + CSS en front cuando passkeys de cliente está habilitado.                                                         |
| <code>displayCustomerLoginFormAfter</code>                            | Renderiza botones/UI: passkey login, reto TOTP pendiente, recovery form.                                                                                |
| <code>displayCustomerAccount</code>                                   | Bloque "Doble factor / Passkeys" en <i>Mi Cuenta</i> .                                                                                                  |
| <code>actionAuthentication</code>                                     | Si el cliente tiene TOTP activo, corta el login y redirige al reto TOTP.                                                                                |

## 5. Endpoint AJAX único

`modules/ecom_passkeytotp/ajax.php` — todas las acciones cliente y admin pasan por aquí. NO se usan controladores front nuevos (regla del proyecto).

## Acciones soportadas

|                                    |                                                         |
|------------------------------------|---------------------------------------------------------|
| <code>generateChallenge</code>     | – GET – emite challenge WebAuthn + datos del usuario    |
| <code>csrf_token</code>            | – GET – refresca el token CSRF                          |
| <code>login</code>                 | – POST – verifica assertion WebAuthn (admin o customer) |
| <code>register</code>              | – POST – guarda credencial WebAuthn nueva               |
| <code>delete</code>                | – POST – borra passkeys (mine / all_admin)              |
| <code>totp_get</code>              | – GET – devuelve si el cliente tiene TOTP activo        |
| <code>totp_set</code>              | – POST – activa TOTP del cliente                        |
| <code>totp_verify</code>           | – POST – verifica código TOTP                           |
| <code>totp_delete</code>           | – POST – desactiva TOTP del cliente                     |
| <code>totp_backup_count</code>     | – GET – cuántos códigos de respaldo quedan              |
| <code>totp_backup_consume</code>   | – POST – consume un código de respaldo                  |
| <code>totp_recover_request</code>  | – POST – envía link de recuperación por email           |
| <code>totp_recover_complete</code> | – POST – usa link + nueva password para reset           |
| <code>updateCustomer</code>        | – POST – recalcula sesión del customer tras login       |

CSRF: header `X-Ecom-Passkeytotp-Csrftoken` o campo `ecom_passkeytotp_csrf`. Verificado en todas las acciones que mutan estado. Acciones de solo lectura ( `generateChallenge` , `totp_get` , `totp_backup_count` , `csrf_token` ) están exentas.

## 6. Convenciones críticas (NO romper)

### 6.1 Nombre de la clase principal con `_`

La clase del archivo `ecom_passkeytotp.php` es `Ecom_Passkeytotp`. **El guión bajo es obligatorio**: PrestaShop resuelve la clase del módulo a partir del nombre de la carpeta ( `ecom_passkeytotp` ) y `class_exists` en PHP empareja case-insensitive pero **NO** normaliza guiones bajos. Sin el `_`, la instalación rompe con:

```
TypeError: get_parent_class(): Argument #1 ($object_or_class) must be an object or a valid class name, bool given
```

Misma regla para los front controllers: `Ecom_PasskeytotpInfoModuleFrontController` , `Ecom_PasskeytotpWebauthnModuleFrontController`. El admin controller ( `AdminEcomPasskeytotpController` ) **NO** sigue esta regla — los Tabs admin usan otra convención.

### 6.2 `_PS_ADMIN_DIR_` antes de `config.inc.php`

`ajax.php` define `_PS_ADMIN_DIR_` antes de cargar `config.inc.php`. Si se hace al revés, PS hidrata el contexto front en vez del admin → `$ctx->employee` queda vacío aunque el admin esté logueado → "No se pudo identificar al empleado".

### 6.3 Login admin con passkey — campos cookie

En `hookActionAdminLoginControllerLoginBefore`, escribir SOLO los 5 campos que escribe

`AdminLoginController::processLogin` nativo: `id_employee`, `email`, `profile`, `passwd`, `remote_addr`. **NO** tocar `id_shop`, `id_shop_group`, `id_lang`, `last_passwd_gen` — son `protected` en `Employee` de PS 8.x → 500.

Antes de `$adminCookie->write()`, llamar `$adminCookie->registerSession(new EmployeeSession())`. Sin esto, el AdminDashboard rebota a AdminLogin → loop infinito.

Redirect con URL absoluta: `$ctx->link->getAdminLink('AdminDashboard')` (no relativa, porque AdminController necesita el token en la URL).

## 6.4 Login Admin en PrestaShop 9 (AJAX)

En PS 9, el firewall de seguridad de Symfony intercepta el formulario de login y falla si no hay contraseña. Para evitarlo, el botón de Passkey en el admin no envía el formulario normal, sino que realiza una petición AJAX a `ajax.php?action=login`. Si la firma es válida, `ajax.php` establece la cookie del empleado y devuelve la URL de redirección al Dashboard.

## 6.5 No usar @ para silenciar excepciones de Db

`@Db::getInstance()->execute(...)` **NO** captura `PrestaShopDatabaseException` en dev mode. Para errores de schema, comprobar antes con `INFORMATION_SCHEMA` (helpers `tableExists()`, `columnExists()`, `columnType()`).

## 6.6 No override/, no controllers front nuevos

Reglas duras del proyecto. Toda integración va por hooks ya registrados o `ajax.php`. Los dos controllers front existentes (`info`, `webauthn`) son la excepción heredada — no añadir más.

## 6.7 Smarty Syntax (Curly Braces)

En archivos `.tpl`, las reglas CSS inline o bloques `<style>` deben tener un espacio después de la llave de apertura `{` o estar envueltos en etiquetas `{literal}`. De lo contrario, Smarty intentará procesar el contenido como una etiqueta de plantilla, provocando un error `SmartyCompilerException`.

# 7. Seguridad WebAuthn

`EcomPasskeytotpWebAuthnHandler::verifyAssertion` valida **todo**:

1. Challenge presente (cookie `ecom_passkeytotp_challenge`).
2. Challenge consumido inmediatamente (anti-replay).
3. `clientDataJSON.type === 'webauthn.get'`.
4. `clientDataJSON.origin` matchea el shop.
5. Challenge en `clientDataJSON` matchea el almacenado (hash\_equals binario).
6. `authenticatorData` mínimo 37 bytes.
7. `rpIdHash` matchea sha256 del host del shop.
8. Flag UP (User Present) presente.
9. Flag UV (User Verified) presente si `ECOM_PASSKEYTOTP_PASSKEY_UV_REQUIRED` ON (admin).
10. `openssl_verify` sobre `authData || sha256(clientDataRaw)` con la public key DER almacenada.
11. `signCount` monotonico (anti-clonado de autenticador).

`saveCredential` valida formato de `credential_id` y `public_key`, comprueba que `openssl_pkey_get_public` acepta la clave, limita a **10 passkeys por usuario**, evita duplicados de `credential_id`.

---

## 8. Cifrado de secretos TOTP

---

`EcomPasskeytotpCrypto` : - AES-256-GCM, IV 12 bytes, tag 16 bytes. - Clave derivada por `hash_hkdf('sha256', _NEW_COOKIE_KEY_, 32, 'ecom_passkeytotp:<domain>')` (fallback a `_COOKIE_KEY_`). - Formato almacenado: `enc:v1:base64(iv|tag|cipher)`. - `decrypt` devuelve `false` si la clave actual no puede descifrar (típico tras reinstall). - En ese caso, `TwoFactorKey::getSecret / getCustomerRow` **borra la fila** y devuelve `false` para que el flujo arranque limpio.

---

## 9. Auto-reparación

---

`hookActionDispatcher` corre en cada request del front y del admin. Comprueba:

1. ¿Existen las 3 tablas críticas ( `webauthn`, `customer_totp`, `employee_totp` )? Si no, llama a `installDB()`.
2. ¿Existe el Tab `AdminEcomPasskeytotp` ? Si no, llama a `installTab()`.
3. ¿Está el flag `ECOM_PASSKEYTOTP_HOOKS_V4` ? Si no, ejecuta `ensureHooks()` (registra los hooks que falten en TODOS los shops).

Esto significa que si un install queda a medias, el siguiente request lo cierra automáticamente. No hay que desinstalar/reinstalar manualmente.

---

## 10. Logs

---

`EcomPasskeytotpLog` : - Se activan con `ECOM_PASSKEYTOTP_DEBUG = 1`. - Errores siempre se loguean (level ERROR) aunque debug esté off. - Saneamiento por blacklist: cualquier patrón `key=valor` con key sensible ( `password`, `code`, `secret`, `token`, `challenge`, `signature`, etc.) se escribe como `key=<redacted>`. - `logContext()` solo registra metadatos: método, host, URI, action, hash de IP. Nunca el body. - Carpeta `logs/` con `.htaccess Deny from all` autocreado en cada escritura.

# Guía para Profesionales de Desarrollo - EcomPasskeytotp

---

## Extensibilidad

---

El módulo sigue el estándar de PrestaShop 8/9. La lógica de WebAuthn está separada en una clase estática `EcomPasskeytotpWebAuthnHandler` para facilitar su mantenimiento.

## Debugging

---

Para activar los logs detallados: 1. Ir a la configuración del módulo. 2. Activar "Enable Debug Logs". 3. Consultar los ficheros en `/modules/ecom_passkeytotp/logs/`.

## Lógica de Login (Admin)

---

Se utiliza un override de `AdminLoginController::postProcess()`. Si el login por contraseña es exitoso, se intercepta el flujo para validar el código TOTP o el reto de Passkey.

## Lógica de Login (Cliente)

---

Se utiliza un Front Controller dedicado `webauthn`. El inicio de sesión se realiza mediante AJAX, el cual actualiza el objeto `Customer` en el contexto y regenera la cookie de sesión.

## Criptografía

---

WebAuthn utiliza algoritmos asimétricos (comúnmente ES256). La llave pública se almacena en Base64 en la base de datos.

## Advertencia de Mantenimiento

---

- **Importante:** Al añadir nuevos hooks o funciones en `addnewfeatures()`, evitar llamar de vuelta a métodos que a su vez invoquen el proceso de instalación para prevenir bucles de recursión (Error 504).

## Fase 2 — Dependencias y entorno EUDI

### Reglas globales

- **CERO Composer.** Cualquier librería se empaqueta en `vendor/internal/<lib>/` con autoload manual.
- Licencias compatibles con AFL-3.0: MIT, BSD-2/3, Apache-2.0, ISC. **Bloqueadas:** GPL/AGPL/LGPL.
- PHP 7.4+ y 8.x — todas las clases deben funcionar en ambos.

### Requisitos PHP nativos (verificar en `addnewfeatures()` y mostrar warning si falta alguno)

| Extensión                              | Mín               | Para qué                                                                                            |
|----------------------------------------|-------------------|-----------------------------------------------------------------------------------------------------|
| <code>openssl</code>                   | OpenSSL 1.1.1+    | Firma/verificación EC P-256/384, RSA, X.509 chain validation.                                       |
| <code>sodium</code>                    | libsodium 1.0.18+ | EdDSA (Ed25519) para firmas modernas.                                                               |
| <code>mbstring</code>                  | nativa            | Manipulación cadenas UTF-8 (disclosures).                                                           |
| <code>json</code>                      | nativa            | Parsing JWT payload, <code>presentation_definition</code> .                                         |
| <code>hash</code>                      | nativa            | SHA-256 / SHA-384 para SD-JWT digest.                                                               |
| <code>xml / dom / xmlreader</code>     | nativa            | Parser LOTL XML (ETSI TS 119 612).                                                                  |
| <code>zlib</code>                      | nativa            | Descompresión de algunos VC firmados.                                                               |
| <code>gmp</code> o <code>bcmath</code> | uno de los dos    | Math grande para verificaciones EC manuales (fallback si OpenSSL no soporta el algoritmo concreto). |

### Librerías embebidas (a copiar dentro de `vendor/internal/`)

#### 1. JWT/JWS/JWK toolkit — implementación propia

**Decisión:** implementación propia minimalista en `classes/Eudi/Crypto/` apoyada en OpenSSL nativo. **Sin librerías externas.** Esto evita problemas de licencia y de dependencias transitivas.

Componentes propios: - `EudiJwt` — encode/decode JWT compacto. - `EudiJws` — verificación JWS firmas detached y compactas. - `EudiJwk` — parsing claves `key=EC|OKP|RSA`. - `EudiJwe` — JWE compacta `ECDH-ES + A256GCM` (única combinación obligatoria). - `EudiSdJwt` — parser SD-JWT con disclosures. - `EudiX509` — verificación cadenas X.509 contra LOTL.

## 2. QR generator

**Librería:** `phpqrcode` (LGPL) → **rechazada por licencia. Alternativa elegida:** implementación QR pura PHP en `classes/Eudi/Util/EudiQr.php`. Solo necesitamos generar QR para URLs cortas (< 256 caracteres). Implementaremos el subset mínimo de ISO/IEC 18004 (versión 1-10, EC level M).

## 3. ASN.1 / DER parser

Para X.509 manual y COSE\_Sign1 (mDoc, fase 2.1). Componente: `classes/Eudi/Crypto/EudiAsn1.php` minimalista.

## Estructura de carpetas

```

ecom_passkeytotp/
├── classes/Eudi/                                ← lógica de negocio EUDI
│   ├── EudiSession.php                        (ObjectModel)
│   ├── EudiPresentation.php                  (ObjectModel)
│   ├── EudiCredential.php                    (ObjectModel)
│   ├── EudiTrustedIssuer.php                (ObjectModel)
│   ├── EudiAttributeMapping.php              (ObjectModel)
│   ├── EudiAuditLog.php                      (ObjectModel)
│   ├── EudiTrustList.php                    (descarga + parse LOTL)
│   ├── EudiRequestBuilder.php                (Authorization Request JWT)
│   ├── EudiVpVerifier.php                    (verifica VP + SD-JWT)
│   ├── EudiAttributeMapper.php               (atributos → Customer/Employee)
│   ├── EudiLoginPerformer.php                (hace login PS legacy/Symfony)
│   ├── EudiQrGenerator.php                  (QR data URI)
│   └── Crypto/
│       ├── EudiJwt.php
│       ├── EudiJws.php
│       ├── EudiJwk.php
│       ├── EudiJwe.php
│       ├── EudiSdJwt.php
│       ├── EudiX509.php
│       ├── EudiAsn1.php
│       └── EudiBase64Url.php
├── controllers/front/
│   ├── eudi_start.php                        (inicia sesión EUDI)
│   ├── eudi_request.php                      (devuelve Authorization Request JWT)
│   ├── eudi_response.php                     (recibe vp_token cifrado)
│   ├── eudi_status.php                       (polling)
│   ├── cron_eudi_trustlist.php                (refresca LOTL)
│   └── cron_eudi_purge.php                    (borra sesiones expiradas)
├── src/Controller/Admin/
│   ├── PasskeyAuthController.php             (ya existe – passkey)
│   └── Eudi/
│       └── EudiAdminAuthController.php        (login admin con EUDI)
├── views/js/
│   ├── eudi.js                               (botón + QR + polling)
│   └── eudi-admin.js                          (mismo flujo en admin login)
├── views/templates/front/
│   └── eudi_popups.tpl                        (modales front EUDI)
├── views/templates/admin/
│   └── eudi_popups_admin.tpl                  (modales admin EUDI)
├── views/css/
│   └── eudi.css
├── docs/
│   ├── EUDI_SCOPE.md                         (Fase 1 – hecho)
│   ├── EUDI_DEPENDENCIES.md                  (Fase 2 – este fichero)
│   ├── EUDI_TECNICO.md                       (Fase 13)
│   ├── EUDI_DESARROLLO.md                    (Fase 13)
│   ├── EUDI_USUARIO.md                       (Fase 13)
│   └── EUDI_TESTING.md                       (Fase 12)
└── vendor/internal/                          ← reservado para libs externas (vacío en v2.0)

```

## Constantes de configuración nuevas ( `Configuration::set` )

| Clave                                          | Tipo              | Default                                      | Descripción                                                 |
|------------------------------------------------|-------------------|----------------------------------------------|-------------------------------------------------------------|
| <code>ECOM_EUDI_ENABLED</code>                 | bool              | 0                                            | Master switch del módulo EUDI.                              |
| <code>ECOM_EUDI_ENABLED_FRONT</code>           | bool              | 0                                            | Botón visible en login cliente.                             |
| <code>ECOM_EUDI_ENABLED_ADMIN</code>           | bool              | 0                                            | Botón visible en login admin.                               |
| <code>ECOM_EUDI_CLIENT_ID</code>               | string            | <code>https://store.com/eudi/verifier</code> | URL del verifier (este módulo).                             |
| <code>ECOM_EUDI_PRIVATE_JWK</code>             | text<br>(cifrado) | <code>"</code>                               | JWK privada del verifier. Generada al instalar.             |
| <code>ECOM_EUDI_PUBLIC_JWK</code>              | text              | <code>"</code>                               | JWK pública correspondiente.                                |
| <code>ECOM_EUDI_REQUIRED_ATTRS_FRONT</code>    | text<br>(json)    | <code>["email"]</code>                       | Atributos solicitados en login cliente.                     |
| <code>ECOM_EUDI_REQUIRED_ATTRS_ADMIN</code>    | text<br>(json)    | <code>["email"]</code>                       | Atributos solicitados en login admin.                       |
| <code>ECOM_EUDI_PURPOSE_FRONT</code>           | string            | <code>'Login en la tienda'</code>            | Texto purpose mostrado al usuario.                          |
| <code>ECOM_EUDI_PURPOSE_ADMIN</code>           | string            | <code>'Login back-office'</code>             | Texto purpose admin.                                        |
| <code>ECOM_EUDI_TRUST_MODE</code>              | enum              | strict                                       | strict / permissive.                                        |
| <code>ECOM_EUDI_TRUST_LIST_URL</code>          | string            | LOTL UE oficial                              | URL XML de la LOTL.                                         |
| <code>ECOM_EUDI_TRUST_LIST_LAST_REFRESH</code> | int               | 0                                            | Timestamp último refresco.                                  |
| <code>ECOM_EUDI_AUTO_REGISTER</code>           | bool              | 0                                            | Si <code>1</code> , crea Customer nuevo si email no existe. |
| <code>ECOM_EUDI_DUPLICATE_EMAIL_POLICY</code>  | enum              | block                                        | block / merge_manual / ignore.                              |
| <code>ECOM_EUDI_RETENTION_DAYS</code>          | int               | 30                                           | Días que se guarda VP token / disclosures.                  |
| <code>ECOM_EUDI_RATE_LIMIT_RESPONSE</code>     | int               | 5                                            | Max responses por IP por minuto.                            |

| Clave                 | Tipo | Default | Descripción                               |
|-----------------------|------|---------|-------------------------------------------|
| ECOM_EUDI_SESSION_TTL | int  | 120     | Segundos de validez de un challenge EUDI. |

## Decisiones técnicas críticas

1. **Generación de claves:** al instalar/activar EUDI, se genera un par EC P-256 ( `prime256v1` ) con `openssl_pkey_new` . La privada se cifra con `EcomPasskeytotpCrypto` antes de `Configuration::set` .
2. **client\_id:** por defecto `https://<shop_url>/eudi/verifier` . En OpenID4VP se permite que el `client_id` sea simplemente la URL HTTPS del verifier (esquema `redirect_uri` ).
3. `request_uri` debe ser HTTPS público y resoluble por wallets (no localhost).
4. `response_uri` mismo dominio HTTPS.
5. **No se usan jwks endpoints públicos:** las JWKs públicas del verifier se incluyen inline en `client_metadata` dentro del JWT firmado.
6. **Privacidad:** `vp_token` se descifra → se valida → se extraen atributos → se guarda **solo** lo aprobado por `presentation_definition` cifrado en `ecom_eudi_presentations` .

## Estado de Fase 2

- Estructura de carpetas creada ( `vendor/` , `vendor/internal/` , `classes/Eudi/` , etc).
- `index.php` blockers añadidos.
- Decisiones técnicas congeladas en este documento.
- **Ninguna dependencia externa** se va a empaquetar — todo PHP nativo + OpenSSL.

# Fase 1 — Scope EUDI Wallet Login

## Decisiones congeladas (versión v2.0 del módulo)

### Rol del módulo

- **Verifier / Relying Party** — sí.
- **Issuer** — no (queda fuera del alcance v2.0).
- **Wallet propio** — no (nunca).

### Estándares y versiones

- **OpenID4VP** — `OpenID for Verifiable Presentations` draft 22 (2026-Q1) como base.
- **SD-JWT VC** — IETF `draft-ietf-oauth-sd-jwt-vc-09` . Formato **obligatorio** en v2.0.
- **mDoc / ISO 18013-5** — fuera de v2.0; previsto v2.1.
- **DID** — soporte mínimo `did:jwk` y `did:web` . Otros métodos quedan fuera.
- **EUDI ARF** — versión 1.5.0 (anclaje formal).

### Modos de uso

- **Same-device**: deep link `openid4vp://authorize?request_uri=...` .
- **Cross-device**: QR generado en el servidor, polling de estado por el front cada 1.5 s.
- Ambos modos obligatorios desde v2.0.

### Casos de uso v2.0

| Caso                            | Atributos solicitados    | Acción                                                                 |
|---------------------------------|--------------------------|------------------------------------------------------------------------|
| Login cliente                   | <code>email</code>       | Autenticar cliente existente o registrar nuevo (configurable).         |
| Login admin                     | <code>email</code>       | Solo si el <code>email</code> mapea a un <code>Employee</code> activo. |
| Verificación de mayoría de edad | <code>age_over_18</code> | Marca <code>customer.age_verified = 1</code> .                         |

### Casos de uso v2.1+ (no se implementa ahora, pero la arquitectura los soporta)

- KYC completo ( `given_name` , `family_name` , `birthdate` , `birth_place` , `nationality` ).
- Empresas ( `legal_entity_id` , `LEI` , `EORI` ).
- B2B onboarding.

### Selective disclosure

- Obligatorio. Solo se procesan disclosures que están en el `presentation_definition` .

- Las disclosures recibidas de más se descartan silenciosamente y se loggean (no se guardan ni se usan).

## Almacenamiento

- VP token y disclosures **cifradas** con `EcomPasskeytotpCrypto` antes de guardar.
- Retención por defecto: 30 días. Configurable en admin.
- Borrado automático vía cron `cron_eudi_purge.php`.

## Trust / confianza en issuers

- Lista UE de Trusted Lists (LOTL) con descarga + verificación de firma.
- Cron diario `cron_eudi_trustlist.php` refresca local cache.
- Modo **estricto** (default) — solo issuers en LOTL.
- Modo **permisivo** (configurable por sysadmin) — acepta issuers fuera de LOTL pero loggea WARNING.  
**No recomendado.**

## Criptografía mínima

- Algoritmos firma del VP: **ES256, ES384, EdDSA** obligatorios. RS256 aceptado pero deprecated.
- Algoritmos JWE response encryption: **ECDH-ES + A256GCM** obligatorio.
- Verificación X.509 chain con OpenSSL nativo (PHP `openssl_x509_*`).
- COSE\_Sign1 (mDoc) — fuera de v2.0.

## Compatibilidad

- PHP **7.4+, 8.x** — obligatorio.
- PrestaShop **8.x y 9.x** — ambos soportados.
- En PS<9, el flujo de login admin con EUDI usa la rama legacy (cookie). En PS9, usa el controller Symfony.

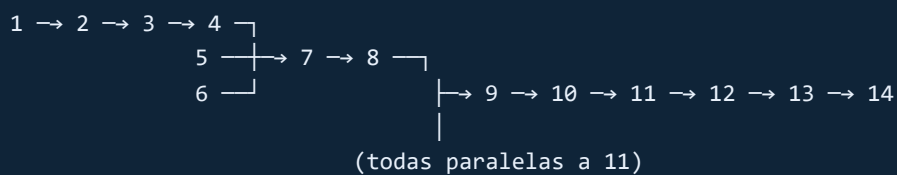
## Reglas del proyecto que aplican

- **CERO Composer**: cualquier librería JWT/JWS/JWE empaquetada en `vendor/internal/` con autoloader manual. Licencia compatible con AFL-3.0 (MIT, BSD, Apache 2.0 OK; GPL no).
- **Sin overrides core**.
- **Sin namespace** en el archivo principal del módulo.
- Headers `Ecom Experts / 2026 / AFL-3.0` en todos los ficheros nuevos.
- `index.php` blocker en cada carpeta nueva.

## Riesgos conocidos y mitigaciones

| Riesgo                                                            | Mitigación                                                                                                  |
|-------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| ARF cambia entre 1.4/1.5/1.6                                      | Pinning explícito a 1.5.0; tests con sandbox UE EUDIW reference.                                            |
| LOTL UE indisponible                                              | Cache local de 7 días con WARNING en admin si no se refresca.                                               |
| Wallet con UA diferente al verifier                               | El protocolo OpenID4VP no requiere igualdad de UA; explícitamente no usar <code>_sf2_meta.u</code> .        |
| Replay del mismo <code>vp_token</code>                            | Tabla <code>ecom_passkeytotp_challenges</code> reutilizada; <code>nonce</code> consumido tras verificación. |
| DOS en <code>/eudi/response</code>                                | Rate-limit por IP (5 req / minuto), reuso de la lógica de <code>customer_totp_recovery</code> .             |
| Email recibido del wallet duplicado en otro <code>Customer</code> | Configurable: bloquear, fusionar (manual), o ignorar. Default: bloquear.                                    |

## Dependencias entre fases



## Nombre comercial

- v2.0 → `Ecom Passkey, TOTP & EUDI` (mismo módulo, nuevo capability).
- Carpeta del módulo: **NO se renombra** (sigue siendo `ecom_passkeytotp`) para no romper instalaciones existentes (regla `project_class_name_underscore`).

## Endpoints nuevos (preview de Fase 5/6)

| Método | Path                                           | Auth             | Función                                                                                 |
|--------|------------------------------------------------|------------------|-----------------------------------------------------------------------------------------|
| GET    | /modules/ecom_passkeytotp/eudi/start           | front<br>anónima | Inicia sesión, devuelve <code>state</code> + <code>request_uri</code> + <code>qr</code> |
| GET    | /modules/ecom_passkeytotp/eudi/request/{state} | anónima          | Devuelve la <code>Authorization Request</code> JWT firmada                              |
| POST   | /modules/ecom_passkeytotp/eudi/response        | anónima          | Recibe el <code>vp_token</code> cifrado                                                 |
| GET    | /modules/ecom_passkeytotp/eudi/status          | front<br>anónima | Polling: estado de la sesión                                                            |
| GET    | /admin{dir}/ecom_passkeytotp/eudi/admin/start  | back-office      | Mismo flujo para admin                                                                  |
| GET    | /admin{dir}/ecom_passkeytotp/eudi/admin/status | back-office      | Polling admin                                                                           |

## Contrato JSON `eudi/start` (preview)

```
{
  "success": true,
  "state": "abc123",
  "request_uri": "https://store.com/modules/ecom_passkeytotp/eudi/request/abc123",
  "deep_link": "openid4vp://authorize?request_uri=https%3A%2F%2F...",
  "qr_data_uri": "data:image/svg+xml;base64,...",
  "expires_in": 120
}
```

## Quién hace qué

- **Backend:** el verifier emite la `Authorization Request` firmada, recibe el `vp_token`, valida cripto y atributos, hace login.
- **Frontend (JS):** muestra QR/deep link, hace polling, redirige al éxito.
- **Wallet (cliente final):** parte del usuario; nunca ejecutamos código en él.

## Glosario rápido

- **VP** = Verifiable Presentation (lo que entrega el wallet).
- **VC** = Verifiable Credential (lo que está dentro del VP, emitido por un issuer).
- **SD-JWT** = Selective Disclosure JWT — formato compacto donde cada claim del VC se hashea por separado y solo se revelan los necesarios.
- **client\_id** = identificador del verifier (este módulo). En OpenID4VP normalmente es la URL https del verifier.
- **nonce** = aleatorio del verifier para anti-replay. Reutilizamos `ecom_passkeytotp_challenges`.
- **state** = identificador opaco que liga la petición con la sesión web del usuario.

# Diagnóstico definitivo PS8 → PS9 — Passkey & 2FA

Fecha: 2026-05-09. Investigación contrastada con docs oficiales PrestaShop 9 y 8, repositorio público y observación del entorno `presta9.gmartos.es` (PS 9.1.1, PHP 8.3.30).

## 1. Lo que cambió entre PS8 y PS9 (fuente: devdocs oficial)

### 1.1 Hooks de login admin **ELIMINADOS** en PS9

Toda la familia legacy fue retirada al migrar el back-office a Symfony Security: -

`actionAdminLoginControllerBefore` - `actionAdminLoginControllerLoginBefore` ← lo usaba el módulo -  
`actionAdminLoginControllerLoginAfter` - `actionAdminLoginControllerForgotBefore/After` -  
`actionAdminLoginControllerResetBefore/After`

**Implicación:** todo el código de TOTP del módulo en `hookActionAdminLoginControllerLoginBefore` **NO se ejecuta en PS9**. Por eso el TOTP "no hace caso" en PS9.

### 1.2 Hooks que se mantienen

- `actionAdminLoginControllerSetMedia` (assets CSS/JS) ✓
- `displayAdminLogin` (HTML extra) ✓

### 1.3 Hooks **NUEVOS** en PS9

- `actionBackOfficeLoginForm` — recibe `form_builder` para añadir campos al form de login.
- `actionEmployeeRequestPasswordResetForm` — formulario de reset de password.

### 1.4 Sesiones y cookies — cambio crítico

- En PS8: `Context::$cookie` ESCRITA por el módulo era válida para autenticar ( `$cookie->id_employee = X;` `$cookie->write();` ).
- En PS9: `Context::$cookie` es **SOLO LECTURA** para datos de autorización. Las modificaciones **no surten efecto**. La sesión Symfony del lado del servidor ( `PHPSESSID` ) es la fuente de verdad.
- Datos custom: `$request->getSession()->set('clave', 'valor')` .

### 1.5 Clase `Employee` para Symfony Security

- En PS8: `PrestaShopBundle\Security\Admin\Employee` implementaba `UserInterface` .
- En PS9: `PrestaShopBundle\Security\Admin\Employee` **ELIMINADA**. La implementa ahora `PrestaShopBundle\Entity\Employee\Employee` (Doctrine entity).
- `EmployeeProvider::supportsClass()` **solo acepta** la clase Doctrine entity. Pasar el `\Employee` legacy a `Security::login()` lanza `UnsupportedUserException` .

## 1.6 EmployeeProvider::refreshUser() — origen del "Tu sesión se ha cerrado por motivos de seguridad"

```
public function refreshUser(UserInterface $user) {  
    if (!$user instanceof DoctrineEmployee) {  
        throw new UnsupportedUserException(...);  
    }  
    return $this->loadEmployee($user->getUserIdentifier(), refresh: true);  
}
```

- `ContextListener` deserializa el token de sesión al inicio del request, llama a `refreshUser($user)`, y si la deserialización dio un user de tipo no-Doctrine o si el `EquatableInterface::isEqualTo()` falla, **invalida la sesión** y muestra el mensaje "Tu sesión se ha cerrado por motivos de seguridad".

## 1.7 Para reaccionar al login en PS9 — patrón oficial

*"If you need to integrate your code with the authentication process you should now rely on the Authentication events described in Symfony's documentation, or inspire yourself from PrestaShop's internal subscriber."*

→ **Symfony EventSubscriber** registrado en `config/services.yml` del módulo con tag `kernel.event_subscriber`, escuchando: - `Symfony\Component\Security\Http\Event\LoginSuccessEvent` — disparado tras autenticación exitosa. - `Symfony\Component\Security\Http\Event\CheckPassportEvent` — para validar credenciales adicionales (TOTP). - `Symfony\Component\Security\Http\Event>LoginFailureEvent` — auditar fallos.

## 2. Por qué cada uno de mis intentos falló

| Intento                                                                                    | Por qué falló                                                                                                                                                                                                             |
|--------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Hook<br><code>actionAdminControllerLoginBefore</code> con <code>redirect</code>            | No se ejecuta en PS9 (hook eliminado).                                                                                                                                                                                    |
| Manipular <code>\$cookie-&gt;id_employee</code> desde <code>ajax.php</code>                | En PS9 la cookie legacy es read-only. Symfony ignora la cookie, lee la sesión Symfony, no encuentra <code>_security_main</code> válido → bota.                                                                            |
| Bootear <code>AdminKernel</code> standalone desde <code>ajax.php</code>                    | Sin <code>Request</code> en <code>RequestStack</code> , los servicios <code>security.*</code> son privados/no instanciables. Probado:<br><code>has_security_helper=false</code> , <code>has_session=false</code> .        |
| Escribir <code>_security_main</code> directo en <code>\$_SESSION['_sf2_attributes']</code> | Sí persiste, pero al deserializar <code>ContextListener</code> llama a <code>refreshUser()</code> y falla por <code>EquatableInterface::isEqualTo()</code> → "Tu sesión se ha cerrado por motivos de seguridad".          |
| <code>PasskeyAuthController</code> Symfony con <code>_anonymous_controller: true</code>    | El controller queda <b>fuera del firewall main</b> , así que <code>Security::login()</code> no encuentra el firewall correcto. Y el router PS9 añade <code>?_token=</code> vacío al generar la URL → 500 al validar CSRF. |
| Cookie HMAC propia para TOTP                                                               | Funcional, pero el guard del dispatcher creaba bucles porque <code>actionDispatcher</code> . Sí se ejecuta en PS9 antes que el routing Symfony, y mi <code>header('Location') + exit</code> cortaba flujos legítimos.     |

## 3. Solución correcta arquitectónica para PS9

### 3.1 Estructura

```
ecom_passkeytotp/
├── config/
│   └── services.yml      ← NUEVO: registra subscribers en el container Symfony
├── src/
│   ├── EventSubscriber/
│   │   └── AdminLoginSubscriber.php ← NUEVO: escucha LoginSuccessEvent
│   └── Controller/Admin/
│       ├── PasskeyAuthController.php ← lo que ya hay
│       └── TotpGateController.php    ← NUEVO: pantalla intermedia TOTP
```

### 3.2 Flujo de login admin TOTP en PS9

```

Usuario mete email+password en /admin{dir}/login
↓ Symfony Security autentica → emite LoginSuccessEvent
↓ AdminLoginSubscriber::onLoginSuccess($event)
  - Si employee tiene TOTP activo y no marcado verified en sesión:
    - $session->set('ecom_totp_pending', $employee->id);
    - $event->setResponse(new RedirectResponse('/admin{dir}/ecom_passkeytotp/totp/gate'));
  - Si no, dejar pasar al dashboard.
↓ TotpGateController::index → muestra form
↓ Usuario mete código de 6 dígitos
↓ TotpGateController::verify
  - TwoFactorKey::verifyCode() → si OK $session->set('ecom_totp_verified', $employee->id)
  - Redirige al dashboard
↓ AdminLoginSubscriber escucha también `kernel.request` para que cualquier acceso al admin
sin TOTP verificado vuelva al gate. Sin bucles porque la propia ruta gate está exentada.

```

### 3.3 Flujo passkey admin en PS9

```

Front login muestra botón "Login con Passkey".
JS hace WebAuthn → POST a PasskeyAuthController::index (ruta Symfony del módulo
__sin__ `_anonymous_controller: true`, dentro del firewall main pero con
`IS_AUTHENTICATED_ANONYMOUSLY` permitido).
PasskeyAuthController:
  - Valida assertion WebAuthn (mismo código de antes).
  - $employeeProvider->loadUserByIdentifier($email) → DoctrineEmployee
  - $security->login($doctrineUser, 'main', null) → LoginSuccessEvent emitido
  - El subscriber al ver passkey-verified marca `ecom_totp_verified` también
    (passkey === factor fuerte, no requerimos TOTP encima).
  - Devuelve JSON con redirect al dashboard.

```

## 4. Pendiente — pasos concretos

1. Crear `config/services.yml` con autowire/autoconfigure y registro del subscriber.
2. Crear `src/EventSubscriber/AdminLoginSubscriber.php` que escuche `LoginSuccessEvent` + `RequestEvent`.
3. Crear `src/Controller/Admin/TotpGateController.php` con dos acciones: `gate` (form) y `verify` (POST).
4. Añadir entradas en `config/routes.yml` para las dos acciones del gate.
5. Reescribir `PasskeyAuthController` para que NO use `_anonymous_controller` (que provoca el 500), sino que esté en el firewall y use `IS_AUTHENTICATED_ANONYMOUSLY`.
6. **NO tocar el flujo PS≤8** (que funciona). Todo el código nuevo se gateará por `version_compare(_PS_VERSION_, '9.0.0', '>=')` o por la sola presencia del subscriber Symfony (que en PS<9 no se carga).

## 5. Fuentes consultadas

- [PrestaShop 9.0 changes — devdocs oficial](#)

- [PR #35983 — Migrate login and authentication in Back Office to Symfony](#)
- [PrestaShop 9 modules services — devdocs](#)
- [example-modules / demodoctrine — services.yml de ejemplo oficial](#)
- [Symfony 6.4 Security events](#)
- Código fuente de `PrestaShopBundle\Security\Admin\EmployeeProvider` (rama develop).